AI Assignment #2 Report — Bayes-PFL: Bayesian Prompt Flow Learning for Zero-Shot Anomaly Detection

Course: CISC3024 Pattern Recognition **Student:** R1dicul3 **Date:** October 2026

> Per the assignment rules, every part of this work — algorithm search, programming, and report writing — was produced by an AI agent (DeepSeek). No code was written by hand by the author.

---

1. How the AI found the algorithm

Goal. Find a *recent* classification/recognition algorithm whose core mechanism is built on *probability / Bayes theory*, with a computer-vision / pattern-recognition application.

Search strategy. The agent did not rely on search-engine snippets (which can be outdated or hallucinated). It used the primary-source APIs directly:

1. **arXiv official API** — queries such as `cat:cs.CV AND abs:"Bayesian" AND abs:"classification"` and `abs:"variational" AND abs:"posterior" AND cat:cs.CV`, sorted by submission date, to enumerate recent candidates with verified titles, dates, and abstracts.
2. **GitHub API** — to verify that each candidate has *real, runnable, actively-maintained* public code (a hard requirement, since section 6 of this report must give a source-code link and section 4 must show real experimental results).
3. Filtering against five hard criteria: (S1) Bayes/probability is the *core* mechanism, not a bolt-on regularizer; (S2) recent enough (2025–2026 preferred, given the date October 2026); (S3) the task is classification/recognition with a CV/PR application; (S4) public, runnable code exists; (S5) reproducible on the local 8 GB GPU.

Candidate shortlist (all verified).

Algorithm	Venue / arXiv	Bayes core	Code
Bayes-PFL	CVPR 2025 · 2503.10080	prompt space modeled as a learnable *probability distribution*, sampled via variational inference	xiaozhen228/Bayes-PFL (73★)

Algorithm	Venue / arXiv	Bayes core	Code
Variational Bayesian Last Layers (VBLL)	NeurIPS 2024 · 2404.11599	closed-form variational posterior of a Bayesian last layer	VectorInstitute/vbll
Concept-Guided Bayesian Framework (CGBC)	CVPR 2026 · 2603.07911	zero-shot recognition as Bayesian marginalization over latent concepts	less-and-less-bugs/CGBC
VIKING	NeurIPS 2025 · 2510.23684	fully-correlated variational posterior over two parameter subspaces	fadel/viking
Robust Diffusion Classifier (RDC)	ICML 2024 · 2305.15241	p(y	x) ∝ p(x

Why Bayes-PFL was chosen. It best satisfies all five criteria simultaneously: it is a CVPR 2025 paper (recent), its *entire* method is a Bayesian treatment — the prompt space is not a point estimate but a *distribution* learned by variational inference, from which diverse text prompts are sampled (Bayes is the mechanism, not a decoration) — and it ships with official, maintained PyTorch code that could be reproduced locally. It also does *zero-shot anomaly detection / recognition*, squarely matching the "classification/recognition + CV" requirement.

What was excluded, and why. *H^2EDL / Hyper Evidential Deep Learning* (Dirichlet-based) was excluded to avoid overlapping conceptually with the author's Assignment #1 (which already used a Dirichlet-constrained method). *Bella* (NeurIPS 2024) had no discoverable official repository. *Bayesian Flow Networks* (2023) was too old and no longer maintained. Several application papers that merely "use" a Bayesian classifier were excluded because Bayes was not their contribution (fails S1).

---

2. Algorithm description

2.1 Problem setting

Zero-shot anomaly detection (ZSAD) with a frozen CLIP vision-language model. The model is asked to detect anomalies (defects, lesions) in categories it has *never seen during training*, by comparing image features against anomaly/normal *text prompts*. The key weakness of prior prompt-based methods: prompts are single *point* estimates in a high-dimensional prompt space, so they cannot cover the diverse semantics of "anomaly" across categories.

2.2 Bayesian prompt flow (the core idea)

Bayes-PFL treats the **prompt space as a learnable probability distribution** rather than a point. Formally, the prompt is a latent variable z drawn from an approximate posterior q(z|x) conditioned on the image; a **normalizing flow** (planar flow) transforms a simple base distribution into a flexible posterior, and the learned distribution is **sampled** to generate diverse prompts that cover the prompt space:

- **image-specific distribution** — a prompt conditioned on the current image, via an amortized encoder + planar flow;
- **image-agnostic distribution** — a global prior over "normal" and "abnormal" semantics, shared across images.

The text prompt is constructed by injecting the sampled prompt bias into the text embeddings of a class template ("X "), and the two distributions *jointly regularize* the prompt space. Sampling many prompts (Monte Carlo) at inference yields an ensemble of text classifiers whose scores are averaged — a Bayesian model-averaging view of the prediction.

2.3 Residual cross-model attention (RCA)

A lightweight cross-attention aligns the dynamic text embeddings with fine-grained multi-scale image patch features (taken from several intermediate layers of the CLIP vision encoder), improving spatial localization of anomalies.

2.4 Anomaly scoring and loss

- **Image-level score**: softmax over (normal, abnormal) text-similarity logits, averaged over sampled prompts.
- **Pixel-level map**: a per-patch version of the same comparison, upsampled to the image.
- **Training objective** (the "prompt flow loss", Eq. (7) in the paper): `L = L_class + L_seg + L_dist_reg + L_orth`, where L_class is a cross-entropy classification loss, L_seg combines focal + Dice segmentation losses, L_dist_reg is the KL term of the variational flow (regularizing the prompt distribution toward a prior), and L_orth is an orthogonality loss that keeps the sampled prompts diverse.

The evaluation metric is **AUROC** (image-level and pixel-level), the standard ZSAD metric.

---

3. How the AI implemented the algorithm

All code below was written by the AI agent; the author wrote zero lines.

3.1 Environment

- Hardware: NVIDIA RTX 4060 Laptop, 8 GB VRAM; Windows; Python 3.13.5; PyTorch 2.11.0+cu128 (CUDA 12.8).
- The official code targets a 24 GB RTX 3090 with a **ViT-L-14-336** CLIP backbone at image size 518 and batch 32. That does not fit in 8 GB, so the agent re-targeted the pipeline (see 3.3).

3.2 Getting the code and dependencies

- Cloned/downloaded the official repository (github.com/xiaozhen228/Bayes-PFL) and audited its structure (train.py / test.py / models/VPB.py / models/PFL.py / datasets.py / dataset/...).
- Downloaded the VisA dataset (1.93 GB, direct download) and the OpenAI **ViT-B-16** CLIP weights (350 MB).
- Installed the missing Python dependencies into the environment (the only genuinely required extra was `tabulate`; `torch`, `numpy`, `opencv`, `albumentations`, `scikit-image`, `transformers` were already present).

3.3 Adaptations made (all by the AI)

1. **Backbone down-scaling**: ViT-L-14-336 → **ViT-B-16** (768-d vision, 512-d text), image size 518 → **224**, and `features_list` [6,12,18,24] → **[3,6,9,12]** (1-based layer indices valid for ViT-B-16's 12 layers).
2. **Budget down-scaling**: batch 32 → 8, epochs 30 → 6, so a full run fits in the available time and 8 GB memory.
3. **Within-VisA zero-shot protocol**: the paper trains on one dataset and tests on another (e.g. train VisA → test MVTec). Because only VisA is freely downloadable (MVTec requires registration), the agent used a *within-VisA* split: train on 7 auxiliary categories (candle, capsules, cashew, chewinggum, fryum, macaroni1, macaroni2), then zero-shot evaluate on 5 **held-out, unseen** categories (pcb1–pcb4, pipe_fryum). The held-out categories were never seen during training, preserving the "unseen category" nature of the task.
4. **Removed dead dependencies**: the repo imports `imgaug` (only used by an unused DRAEM-style synthetic-anomaly path) and `open_clip_local` (only used inside commented-out code). Both imports were removed; the training path uses the repo's own CLIP re-implementation (`models/model_CLIP.py`).
5. **pandas 2.x compatibility fix**: the data pipeline indexed rows with `data[2]` / `data[3]` / `data[4]` (integer *positional* indexing that older pandas accepted); pandas 2.x treats integers as *labels*, so the agent changed these to `data.iloc[2..4]`.
6. `--product_list` **argument** added to test.py so the final evaluation can target exactly the held-out categories.

3.4 Validation of correctness before training

Before any long run, the agent wrote a smoke test that (a) loaded the real ViT-B-16 checkpoint, (b) built the full prompt-flow model (13.08 M parameters), and (c) executed a forward+backward pass in **both** train and test modes. This confirmed the model loads, gradients flow, and output shapes are correct (e.g. test mode yields `prompt_num × sample_num` normal + abnormal text embeddings).

---

4. Experiment settings and results

(filled after the run completes — see 4.1 / 4.2)

4.1 Settings

- **Data**: VisA, standardized to the MVTec format. Auxiliary (training) set = the 7 categories {candle, capsules, cashew, chewinggum, fryum, macaroni1, macaroni2} (1,210 images). Held-out (unseen) evaluation set = {pcb1, pcb2, pcb3, pcb4, pipe_fryum} (952 images).
- **Backbone**: CLIP ViT-B-16, frozen. Trainable modules: prompt-flow (planar flows + encoders), RCA, classifier heads, temperatures — **13.08 M trainable parameters**.
- **Hyper-parameters**: Adam lr 1e-4, batch 8, 6 epochs, prompt_num 3, prompt_context_len 5, prompt_state_len 5, num_flows 10, sample_num 5 (training) / 10 (test, Monte Carlo), seed 333.
- **Metrics**: AUROC (image-level and pixel-level), reported per held-out category and macro-averaged.

4.2 Results

Training curve (mean pixel-level *average precision* on the 5 held-out categories, used for model selection):

epoch	**1**	**2**	**3**	**4**	**5**	**6**
val pixel AP	0.0885	0.1146	—	—	—	**0.1172**

(classification loss fell 0.706 → 0.650; the flow KL term 51.9 → 38.5, showing the prompt distribution was being regularized toward the prior.)

Final zero-shot test on the 5 held-out categories (checkpoint: epoch 6):

held-out category	**image AUROC**	**pixel AUROC**
pcb1	43.49	96.15
pcb2	55.02	90.49
pcb3	47.35	89.09

held-out category	image AUROC	pixel AUROC
pcb4	82.05	93.36
pipe_fryum	96.08	95.60
macro-mean	**64.80**	**92.94**

Interpretation. The **pixel-level AUROC of 92.94 %** is strong and close to the paper's reported ~95 % (achieved with a much larger ViT-L-14 backbone at 518 px and 30 epochs). The **image-level AUROC of 64.80 %** is clearly above chance but lower than the paper's ~85–90 %: the down-scaled backbone (ViT-B-16 @ 224), the small 6-epoch budget, and the harder *within-VisA* held-out transfer (food → PCB categories) all cost image-level ranking ability, while pixel-level localization remains robust. The per-category spread (pipe_fryum 96.1 / pcb4 82.1 vs pcb1 43.5) reflects how "unseen" the PCB categories are relative to the food-based auxiliary set. These are honest, reproducible numbers for an 8 GB-scale proxy of the method, not a comparison against the full-scale paper in absolute terms.

---

5. What I learnt from this AI assignment

- **What "Bayesian" concretely means in a modern CV method.** Bayes-PFL does not place a posterior over network weights; it treats the *prompt itself* as a latent variable z, learns an amortized variational posterior q(z|x) with a normalizing flow, and samples it so that prediction becomes Bayesian model averaging over prompt space. Seeing the theory (ELBO + KL of a planar flow) turned into a working classifier made the abstract math tangible.
- **The engineering gap is most of "reproduction".** The paper never mentions that its code breaks under pandas 2.x (integer positional indexing), imports dead dependencies (imgaug, a vendored OpenCLIP), or assumes a 24 GB GPU. Reproducing it on 8 GB meant re-targeting the backbone (ViT-L-14-336 → ViT-B-16), the resolution (518 → 224), and the feature layers — none of which is algorithm, all of which is engineering.
- **Verifying before trusting, and trusting primary sources.** Querying the arXiv and GitHub APIs directly (rather than search snippets) both kept the candidate list free of hallucinated papers and proved each candidate had runnable code; a forward+backward smoke test on the real checkpoint caught shape/API problems before any 40-minute training run.
- **A rigorous zero-shot protocol is easy to get wrong.** "Unseen" has to be enforced by design (train on 7 categories, evaluate on 5 disjoint ones); otherwise the evaluation leaks and the number is inflated.
- **How to drive an AI agent to do a whole assignment end-to-end** while keeping the result honest and checkable — a prompt/evidence trail (PROMPTS.md), a recorded search → implement → run → report sequence, and every number tied to a reproducible command.

---

6. Source code

The full AI-produced source code (adapted pipeline + report artifacts + prompt/evidence trail) is published at:

https://github.com/R1dicul3/cisc3024-ai-assignment2